

Comprehensive FCR & Batch Analytics Review

An technical audit and optimization strategy for poultry batch analytics, metric precision, and backend calculations.

1. Current FCR Computation & Data Flow

Feed Conversion Ratio (FCR) is currently calculated in the Farmer module using the standard definition of feed used ÷ bird weight gained. The active calculation (`batchController.ts:1469-1500`) follows this logic:

- **Initial total weight:** `initial chicks × 0.05 kg`
- **Current total weight:** `latest average weight × current chicks`
- **Total weight gained:** `max(0, current total weight - initial total weight)`
- **FCR:** `total feed consumption ÷ total weight gained`

Data Pipeline Sequence

1. **Feed Entry:** Feed is recorded via an expense record (`expenseController.ts:495`).
2. **Consumption Record:** A corresponding `FeedConsumption` row is created (`schema.prisma:1319`).
3. **Aggregation:** Batch analytics sums all feed quantities across the target timeframe.
4. **Weight Retrieval:** The latest `BirdWeight` row (`schema.prisma:1334`) is fetched by date.
5. **Flock Adjustment:** Mortality rows (`schema.prisma:1240`) are subtracted from the initial flock to determine the current bird count.
6. **API Response:** The API returns calculated metrics (`batchRoutes.ts:50`), which the frontend rounds to two decimal places (`OverviewTab.tsx:138`).

2. Critical Calculation Bugs

Partial Sales Inflate FCR Massively (Highest Risk)

When birds are sold, a `SLAUGHTERED_FOR_SALE` mortality row is created, subtracting them from `currentChicks`. Because the FCR denominator uses `(avgWeight × currentChicks)`,

the weight of the sold birds vanishes, but the feed they consumed remains in the numerator.

- **Impact:** FCR reads artificially high, getting worse with every partial sale. For example, selling 500 out of 1,100 birds can turn a true FCR of 1.49 into a displayed FCR of 2.80.
- **Fix:** Total produced weight must include previously sold birds: `totalLiveWeight = (avgWeight × currentChicks) + totalSalesWeight`.

Incomplete After-Sales FCR

The `fcrAfterSales` formula ($\text{total feed} \div (\text{total sold weight} - \text{initial total chick weight})$) only renders when a batch is marked `COMPLETED`. However, it completely ignores birds that were still present when the batch closed, the weight of birds that died, and incorrectly subtracts the initial weight of all placed chicks rather than just the sold ones (`batchController.ts:1524`).

Null FCR on Closed Batches

Closing a batch records remaining birds as `BATCH_CLOSURE` mortality. Because the FCR queries only exclude `SLAUGHTERED_FOR_SALE`, `BATCH_CLOSURE` drives `currentChicks`, `currentWeight`, and `weightGain` to zero, resulting in a null FCR.

Population Mismatch

Initial weight is calculated using every bird placed ($\text{initialChicks} \times 0.05$), while current weight uses survivors only. While this accurately excludes the weight of dead birds (which correctly penalizes FCR), the initial 50g deduction is applied to birds that are no longer in the denominator. The mathematical impact is minor (~0.25%) but technically imprecise.

3. Data Integrity & State Management

Defect Area	Root Cause	Systemic Impact
Stale Weight Records	<code>latestWeight</code> pulls newest <code>BirdWeight</code> without date-aligning against feed logs.	Additional feed entries skew FCR upward if weigh-ins are delayed.
Sale Edits & Deletions	Editing sales ignores generated weight rows; deleting sales	Desynchronized flock counts and ghost weight metrics.

Defect Area	Root Cause	Systemic Impact
	orphans them (<code>salesController.ts:1056</code>).	
Detached Feed Records	<code>FeedConsumption</code> lacks <code>expenseId</code> link (<code>expenseController.ts:890</code>).	Deleting feed expenses leaves consumption rows intact, distorting FCR.
Feed Units	<code>FeedConsumption.quantity</code> lacks unit validation (assumes kg).	Inputs recorded in bags/sacks cause extreme FCR underreporting.
Unbounded Ranges	Queries aggregate all records without bounding to <code>batch start/end dates</code> .	Out-of-window entries corrupt lifetime batch performance stats.

4. API & UI Inconsistencies

- **Mismatched Mortality Rules:** The batch detail API calculates current birds as `initial - sale mortality - all other mortality`. The dashboard performance API calculates it as `initial - mortality (excluding SLAUGHTERED_FOR_SALE)`. This discrepancy causes the same batch to show different metrics across the app.
- **Layer Batches in Dashboard:** The batch detail page correctly restricts FCR to Broilers. The dashboard performance endpoint does not (`dashboardController.ts:1146`), calculating invalid FCRs for Layer batches.
- **Type Mismatches:** The shared API type (`index.ts:1400`) expects `avgWeight` but the backend returns `currentAvgWeight` and misses fields like `totalFeedConsumption` and `fcrAfterSales`.
- **Minor UI Inconsistencies:** "Net Result" uses `Math.abs()` and colors for sign, while "Per Broiler" shows the raw negative value. Also, the 50g day-old chick weight is hardcoded, whereas real broiler chicks range from 40–45g.

5. Advanced Industry Metrics

To align with modern poultry benchmarking, two additional metrics should be introduced to the reporting suite:

Corrected FCR (cFCR)

Heavier birds naturally convert feed less efficiently. Comparing a flock sold at 1.5 kg to one sold at 2.2 kg using raw FCR is misleading. cFCR normalizes flocks to a reference standard.

$$\text{cFCR} = \text{FCR} - \left(\frac{\text{ActualWeight}}{\text{kg}} - \frac{\text{TargetWeight}}{\text{kg}} \right) \times 10 \times \text{correctionFactor}$$

Note: The reference weight should be configurable per region (e.g., Nepal target ~1.8–2.0 kg vs. EU/US ~2.2–2.4 kg).

European Production Efficiency Factor (EPEF)

EPEF combines mortality, body weight, age, and FCR into a single unified performance score (300–400+ represents strong performance).

$$\text{EPEF} = \frac{\text{Liveability} \%}{\text{AvgWeight} \times \text{Age} \times \text{FCR}} \times 100$$

6. Recommended Action Plan

Step 1: Immediate Core Fixes

Patch the partial-sales calculation bug in `batchController.ts` by adding `totalSalesWeight` to the total live weight equation. Correct `BATCH_CLOSURE` handling to ensure historical records persist after batch completion.

Step 2: API & Calculation Alignment

Standardize mortality definitions across dashboard and batch controllers. Restrict FCR calculations strictly to Broiler batch types across all endpoints.

Step 3: Schema & Data Integrity Safeguards

Enforce explicit weight units on feed entries (`kg` vs `bag`). Add foreign key `expenseId` to `FeedConsumption` to maintain cascade operations when expenses are modified or removed.

Step 4: Analytics Enhancement

Roll out EPEF and cFCR calculations across the dashboard and batch overview tabs, backed by configurable target weights in regional workspace settings.